EXTENDING AN AGENT · THE WHY

An agent by itself can talk. Extending is what lets it act.

Extending means wiring the agent to anything beyond its own reasoning — APIs, models, MCP servers, peer agents, automation flows, even a browser. The conversation is the surface; the extensions are the hands.

TOOLS

Connectors, REST, MCP

The agent calls out to systems you point it at — yours, vendors', MCP servers. Each tool is a verb the model can choose to invoke.

MODELS

Prompts & Foundry models

Bring your own model from Microsoft Foundry. Pick the model that fits the job; use it inside a prompt tool.

PEERS

A2A & agent flows

Hand off to other agents — yours, third parties — or run a flow that takes structured action on the user's behalf.

COMPUTE SURFACES

Computer-use agents

AUTOMATION

Agent flows

WHEN THINGS GO WRONG

Error handling

EXTENSION IS THE AGENT'S SURFACE AREA

Each extension earns its slot by being small, testable, and governable. Pick the extension that matches the answer's blast radius.

CUSTOM CONNECTOR · ANATOMY

A custom connector is a contract the agent holds an API to.

A **custom connector** wraps a REST API into a typed contract — one or more actions, one auth scheme, one set of parameters. Once registered, the connector behaves like a first-class Power Platform citizen and an agent calls it as a tool.

WHAT GETS WRAPPED

Four parts, one wrapper.

HOST
Base URL + region.

AUTH
API key or OAuth.

ACTIONS
Endpoints, params, output.

POLICY
DLP, throttling.

WATCH FOR

A connector defines the surface, not the data

Two “GetCustomer” actions on different connectors look the same to the model until you wire output schemas. Schema discipline is half the difference between a working agent and a hallucinating one.

TWO WAYS TO BUILD ONE

Empty · or from a definition.

FROM BLANK

Wizard — host, auth, action, sample-response.

Best when the API has 1–5 endpoints you can describe by hand.

FROM A DEFINITION

Postman / OpenAPI — import, fix up.

Best when the OpenAPI file is real and you trust the team that wrote it.

RULE

Name actions as verbs. Describe what runs, not what URL the team remembers.

“GetCustomer” beats “getCustomerById-v2”. The model is the audience; the engineer is the author.

REST API ACTIONS · WHEN

A REST API action is the smallest unit an agent can call.

A **REST API action** is one verb on one endpoint, bound to a connector. The action carries request shape, response shape, and an error contract. The agent invokes it the same way it invokes anything else.

FORM A · THE SHAPE A MAKER WRITES

VERB

GetCustomer

PATH

`GET /customers/{id}`

PARAMETERS

id : string, required

RESPONSE

200 — Customer { name, status, tier }

Stored in the connector. Reused across agents. Easy to version, easy to retire.

FORM B · DESCRIBED INLINE (PREVIEW)

Point Copilot Studio at a host, give it an OpenAPI file or a prompt description, pick the endpoints and methods, and the action is bound without leaving the agent canvas.

WHEN THIS FITS

- **An OpenAPI doc is already in flight**
- **You want a quick, single-purpose tool**
- **Certification and review aren't the goal**

Lives with the agent. Promote later only if reuse makes business sense.

PROMOTE ONLY WHEN REUSE JUSTIFIES IT

Inline actions are great for a first draft. Real reuse and lifecycle earn the move to a published connector.

POWER PLATFORM · CONNECTORS & PLUGINS

A connector is a proxy; a plugin is a typed package of related actions.

In Power Platform, a **connector** wraps an API; a **plugin** ships that connector plus its actions, descriptions, and a contract the agent can read. In Copilot Studio the same artefacts show up as tools.

STANDARD CONNECTORS

Shipped with the product

Outlook, Teams, SharePoint, Excel Online — first-party Microsoft surfaces with admin controls already in place.

PREMIUM CONNECTORS

First & third party, paid

ServiceNow, Jira, Zendesk, Salesforce. Licensed per user; gates exist, plan for them.

CUSTOM CONNECTORS

You write the wrapper

Wrap any REST API. Publish privately within the tenant, or certify publicly via Partner Center.

PLUGINS · CERTIFIED PACKAGES

A connector plus its agents

A plugin groups a connector, actions, descriptions and an agent persona — shippable as one unit, certifiable through Partner Center.

AGENTS SEE THEM ALL AS TOOLS

A connector you don’t expose is an action the agent will never pick. Plan the toolset; don’t accumulate it.

MODELS · FOUNDRY & CUSTOM PROMPTS

Copilot Studio lets you bring your own model and use it inside a prompt tool.

Microsoft Foundry hosts a model catalog you can deploy — OpenAI, Anthropic, Mistral, Meta, your own fine-tunes. The **prompt tool** in Copilot Studio is where you swap models, write system prompts, and bind inputs and outputs.

WHERE THE MODELS LIVE

Microsoft Foundry

· Catalog of foundation and fine-tuned models

· One endpoint per deployment

· Pay-per-token or provisioned

· Admin controls live upstream

You bring Foundry; Copilot Studio borrows it.

BINDING

WHERE THE MODEL IS USED

A prompt tool in the agent

MODEL

Pick from a Foundry deployment.

SYSTEM PROMPT

Stable instructions, tone, guardrails.

INPUT · OUTPUT

Bound from variables; passed as JSON.

TEMPERATURE · SAFETY

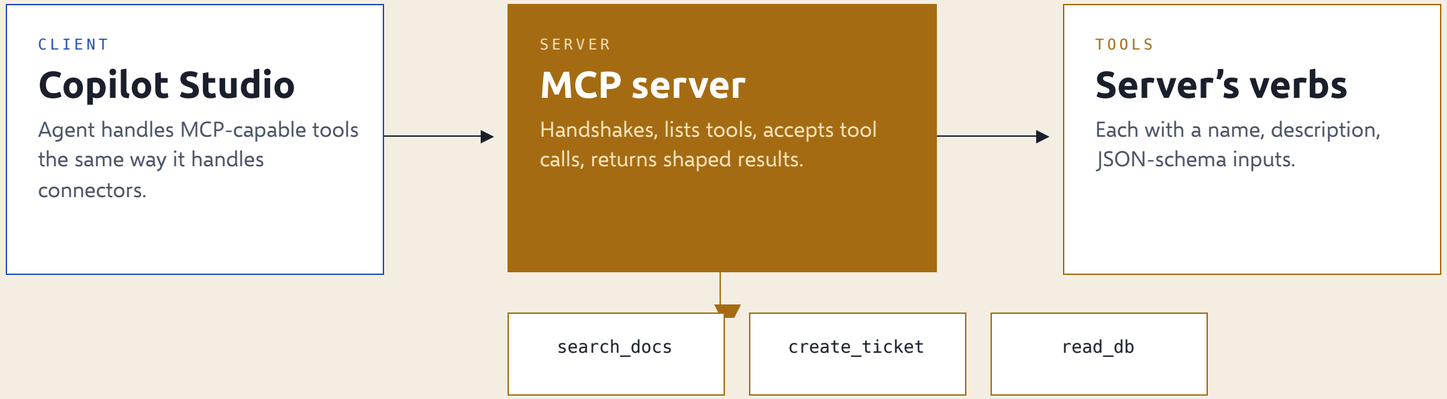
Per-tool settings and moderation.

A prompt tool is the right place to **specialise**. The full conversation stays in the orchestrator model; the tool does the work you actually care about.

MCP · THE PROTOCOL

MCP is a shared contract between agent and tool server.

The **Model Context Protocol** is an open standard. Servers expose tools with names, descriptions and inputs; clients discover them, negotiate, and call them. Copilot Studio speaks MCP on the client side.



WHY THIS IS DIFFERENT

A custom connector is yours. An MCP server is somebody else’s vocabulary you opt into.

You don’t author tools; you bind to ones that already exist and trade descriptions for behaviour.

WHAT THE PROTOCOL ACTUALLY DOES

Three jobs — in order, every call.

01 HANDSHAKE

Negotiate capabilities & auth once.

02 LIST · CALL

Discover tools, invoke by name with JSON args.

03 STREAM · ERRORS

Return shaped output, surface errors as code + message.

WHERE IT TENDS TO LAND

Vendor toolkits, internal data planes, anything protocol-shaped — bound in minutes.

Bring-your-own-server beats writing actions one by one.

WHERE MCP FITS

MCP sits between the reasoner and the thing it calls.

The agent thinks in verbs. MCP is the layer that turns **a verb** into **a request** the server already understands.

LAYER 5

Reasoning model

Picks the verb, fills the inputs, reads the output. Lives in Copilot Studio.

LAYER 4 · ORCHESTRATOR DECIDES “MCP CAN ANSWER THIS”

Generative orchestration

LAYER 3 · THE CONTRACT

Model Context Protocol — JSON-over-HTTP, schema-bound.

LAYER 2

A server you don’t own

LAYER 1

Vendor APIs, files, databases — whatever the server wraps.

FIVE LAYERS, NO HORIZONTAL MOTION

Each layer talks only to its neighbours.

ABOVE MCP

Respects the model’s choices.

The agent doesn’t know MCP exists — it sees a list of tools.

AROUND MCP

Orchestrator decides when to call.

The MCP server doesn’t choose what to do — it answers when asked.

BELOW MCP

Server shields vendor messiness.

Auth details, retries, schema drift — all hidden from the agent.

PORTABILITY IS THE POINT

Swap the server, keep the agent. No rewrite.

A2A · AGENT TO AGENT

A2A lets one agent call another — across tenants or platforms.

Agent2Agent (A2A) is the open protocol one agent uses to discover and call a peer. Each peer exposes an **agent card** — the description a caller needs to decide if it's the right specialist.

CALLER

Your agent

Decides a peer is a better answer for this turn.

DISCOVERY

Fetch agent card

Auth, skills, endpoint — reads like a typed API spec.

CALL

Send task

JSON payload, may be streaming, may be long-running.

PEER REPLIES

Result with shape

Artifact, status, optional follow-up tasks.

AGENT CARD

A typed description

Skills, inputs, outputs, auth, endpoint URL. Caller reads this before invoking.

TASK LIFECYCLE

Submitted · working · done

Polled or streamed. Long-running tasks return IDs first, results later.

ARTIFACTS

Payload after

Files the agent generated; URL + content type.

A2A COMPLEMENTS MCP

MCP is what an agent calls. A2A is who an agent talks to.

Same agent can do both: use MCP to fetch documents, use A2A to defer a hard sub-problem to a peer that knows the domain.

MULTI-AGENT · FOUR PATTERNS

Multiple agents — the shape is the design decision.

A **multi-agent** system is more than one agent working on the same conversation. The wiring between them is the architecture — pick the smallest one that fits the work.

PATTERN 1

Orchestrator + specialists

One parent, several children. Parent owns the conversation; children own one job each.

Works for: "answer, but write the report late."

PATTERN 2

Peer to peer

Any agent can call any other via A2A. No fixed parent. Discovery is the routing layer.

Works for: marketplace of specialists.

PATTERN 3

Sequential pipeline

A → B → C. Each agent refines the work for the next. Tail of the chain talks to the user.

Works for: research → write → review.

PATTERN 4

Magazine of capabilities

Loose pool; any agent can ask any other. Routing is fuzzy, by topic or by skill match.

Works for: open assistants with many modes.

A single agent is the answer more often than you think. Split only when ownership, blast radius, or lifecycle cleanly differ.

HANDOFF · WHEN ONE AGENT LETS GO

A handoff is an explicit transfer of the conversation — not a side comment.

A **handoff** is when an agent decides another agent owns the rest of the turn. Context travels with it; the original agent stops responding. Three shapes show up most.

SHAPE A · TOPIC TRIAGES

Triage by topic

A router agent classifies the query and hands off to a topic-specialist. Decision is local, fast, narrow.

SHAPE B · ONE ASKS ANOTHER

Delegation, not transfer

An agent calls a peer to do a sub-task, then continues. The peer returns; ownership does not move.

SHAPE C · ESCALATION

Out of scope → human

When the agent can't help safely, hand off to a person with a transcript and the variables you collected.

CONTEXT THAT TRAVELS

Three things should follow the handoff.

01
The user's last message, verbatim.

02
Variables the agent already filled.

03
A reason for the transfer the user can read.

A handoff without a reason reads like **a dropped call**. Tell the user where the conversation went and why.

AGENT FLOWS VS TOPICS

Topics handle the conversation; flows handle the work.

Both can do “things.” The split is what **decides**. A **topic** decides contextually; an **agent flow** decides structurally.

TOPIC

A conversational unit

Trigger phrases decide when it runs; conditions and branches route inside it; answers live at the leaves.

GOOD FOR

- **Open conversation with branching**
- **Slot-filling and entity capture**
- **Adaptive cards and clarifying questions**
- **System events (escalation, fallback)**

SHAPE

Trigger phrases → conditions → branches → answer

AGENT FLOW

An automation unit

Triggers run it; actions call systems; branches are explicit. Inputs and outputs are typed. It exists to do the thing the topic delegated.

GOOD FOR

- **Multi-step structured work**
- **Deterministic connect-and-mutate chains**
- **When “if X then Y else Z” beats judgement**
- **Reusing Power Platform without a premium license**

SHAPE

Trigger → actions → conditions → result — same as Power Automate

A topic delegates; a flow delivers. Mix them — the topic decides, the flow executes, the user never sees the seam.

BUILDING AGENT FLOWS · IN PRACTICE

Build the flow like you'd build a function: small inputs, clear outputs, tested alone.

The agent triggers it; the flow returns one shape and one outcome. Treat it as a unit you can reuse, retest, replace.

01

NAME THE WORK

Define one job

A flow does one thing — "create ticket," not "triage this customer."

02

SHAPE I/O

Pin inputs and outputs

Type them; default values; optional vs required.

03

TRIGGER

Pick a trigger

"Run from an agent" or an event — never both for the same flow.

04

ACTIONS · BRANCHES

Wire the chain

Keep branches explicit. Avoid orphan branches — they hide in previews.

05

CONFIGURE EACH FAILURE

Add error branches

Each action gets a fail path. The flow returns a clear "this didn't run".

06

TEST ALONE

Run from the test pane

Before the agent ever calls it. If it can't pass alone, it won't pass through an agent.

THE RULE OF THUMB

If the flow is bigger than the topic that calls it, split the topic. A flow is the muscle; a topic is the handshake.

A COMPUTER-USE TOOL

When there is no API, the agent drives the screen.

A **computer-use agent** interacts with a Windows browser or desktop the way a person would: clicks, types, screenshots, repeats. Use it only when no API exists — with a clear blast radius.

GENERAL AVAILABILITY

Shipped GA in Copilot Studio, 2026.

SHAPE

Browser or full desktop — scoped to a session.

TRUST

A tool of last resort — put it behind a confirm step.

ONE LOOP, FOUR LEGS

Perceive · decide · act · verify.

01 PERCEIVE

Take a screenshot, parse the DOM

Build a working model of what is on screen right now.

02 DECIDE

Pick the next action

Click, type, scroll, wait. Plan with the goal in mind.

03 ACT

Drive input devices

Real keys, real clicks, in a sandboxed session.

04 VERIFY

Re-screenshot, retry, or stop

Same loop each turn; bail on stuck states.

WHEN THIS EARNS ITS SLOT

· The system has no API; the legacy UI is the only surface.

Be conservative, scoped, observable. Capture each session for auditing.

ALMOST NEVER THE FIRST CHOICE

If a connector, an MCP server, or a flow can do it — use that.

Computer-use is for the last ten percent, not the first ninety. It is also the easiest tool to misuse — budget its risk accordingly.

ERROR HANDLING · WHEN TOOLS FAIL

Every extension is an assumption about a system that can refuse.

A network will 500. A model will return empty JSON. A peer will time out. **Design for the refuse**; the work that's left when one of those happens is the real product.

TRANSIENT

Timeouts · 429 · 503

Retry, with backoff. The system will probably say yes next turn.

SEMANTIC

200 with empty data

Define success; design for "no rows" or "no answer" explicitly.

STRUCTURAL

Auth · 4xx · schema

Don't loop — re-route, escalate, or hand off.

FOUR PATTERNS, IN ORDER OF ESCALATION

What the agent does next.

PATTERN 1

Retry with backoff

Same call, three attempts, exponential delay.

PATTERN 2

Fall back to a different tool

A second source; degraded but truthful.

PATTERN 3

Hand off to a peer

A specialist that knows the failure mode.

PATTERN 4

Escalate to a human

Tell the user clearly. Carry the transcript.

TELEMETRY IS PART OF THE ERROR STORY

Log every refuse with who, what, when, why. An agent whose failures are observable is fixable.

LAB · INTEGRATE & EXTEND

Bind four extensions. Then make them argue.

Eight steps, ~90 minutes. The lab ends with a peer review — one extension asks another; both fail gracefully. No hero shots; watch how the parts talk.

01 — 12 MINUTES
Connect a custom connector
Wrap a sample REST API; bind two actions to your agent.

02 — 10 MINUTES
Add an MCP tool
Pick a sample server; confirm handshake, list tools.

03 — 15 MINUTES
Build an agent flow
Wrap the deterministic part; typed inputs and outputs.

04 — 12 MINUTES
Hand off to a peer
Make or expose an A2A peer; configure hand-off.

05 — 10 MINUTES
Bind a Foundry prompt
Pick a model; write the system prompt; wire inputs.

06 — 10 MINUTES
Add error branches
Force a 500; confirm the failure path runs cleanly.

07 — 10 MINUTES
Test with a low-permission user

08 — 10 MINUTES
Peer review the seams

SUCCESS SIGNALS

What “extensions wired” actually looks like.

SIGNAL 1
Four extensions, zero surprises
Each can be invoked alone; each returns a typed shape.

SIGNAL 2
Peer knows the reason
A user can see why their request went to a different agent.

SIGNAL 3
Failure reads as a sentence
No raw codes; the user sees “this didn’t run, here’s why.”

SIGNAL 4
Logs name the seam
Telemetry tells you whether the answer came from a tool, a model, or a peer.

THE RULE

Extensions earn their slot by being observable. If you can’t draw the seam, the agent will struggle with it.